Status	Finished
Started	Friday, 9 October 2026, 12:00 PM
Completed	Friday, 9 October 2026, 12:02 PM
Duration	2 mins 47 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to sort a list of elements using the merge sort algorithm.

For example:

Input	Result
5 6 5 4 3 8	3 4 5 6 8

Answer: (penalty regime: 0 %)

```
def merge_sort(arr):
    # Base case: a list with 0 or 1 elements is already sorted
    if len(arr) <= 1:
        return arr

    # 1. Divide: Split the list into two halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    # 2. Conquer & Combine: Merge the two sorted halves
    return merge(left_half, right_half)


def merge(left, right):
    sorted_arr = []
    i = j = 0

    # Compare elements from both halves and merge them in ascending order
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_arr.append(left[i])
            i += 1
        else:
            sorted_arr.append(right[j])
            j += 1

    # Append any remaining elements left over from either half
    sorted_arr.extend(left[i:])
    sorted_arr.extend(right[j:])

    return sorted_arr


# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    # Read the number of elements (e.g., 5)
    n = int(input())

    # Read the elements as a list of integers
    elements = list(map(int, input().split()))

    # Run the Merge Sort algorithm
    sorted_elements = merge_sort(elements)

    # Print the result separated by spaces
    print(*(sorted_elements))

```

	Input	Expected	Got	
✓	5 6 5 4 3 8	3 4 5 6 8	3 4 5 6 8	✓
✓	9 14 46 43 27 57 41 45 21 70	14 21 27 41 43 45 46 57 70	14 21 27 41 43 45 46 57 70	✓

	Input	Expected	Got	
✓	4 86 43 23 49	23 43 49 86	23 43 49 86	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Print the *sorted array*.

Example 1:

```
Input:
6
1 1 2 2 2 3

Output:
3 1 1 2 2 2

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
```

Example 2:

```
Input:
5
2 3 1 3 2
Output:
1 3 3 2 2
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
```

Example 3:

```
Input:
9
-1 1 -6 4 5 -6 1 4 1
Output:
5 -1 4 4 -6 -6 1 1 1
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

For example:

Input	Result
6 1 1 2 2 2 3	3 1 1 2 2 2
5 2 3 1 3 2	1 3 3 2 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

def frequency_sort(nums):
    # 1. Count the frequency of each number
    freq = Counter(nums)

    # 2. Sort the array using a custom key:
    #    First sort criteria: freq[x] (increasing frequency order)
    #    Second sort criteria: -x (decreasing value order if frequencies tie)
    nums.sort(key=lambda x: (freq[x], -x))

    return nums

# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    try:
        # Read the number of elements
        n = int(input())

```

```
        # Read the elements as a list of integers
        nums = list(map(int, input().split()))

        # Perform custom frequency sorting
        result = frequency_sort(nums)

        # Print the result separated by spaces
        print(*(result))
    except (IOError, ValueError):
        pass
```

	Input	Expected	Got	
✓	6 1 1 2 2 2 3	3 1 1 2 2 2	3 1 1 2 2 2	✓
✓	5 2 3 1 3 2	1 3 3 2 2	1 3 3 2 2	✓
✓	9 -1 1 -6 4 5 -6 1 4 1	5 -1 4 4 -6 -6 1 1 1	5 -1 4 4 -6 -6 1 1 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order. You read an list of numbers. You need to arrange the elements in ascending order and print the result. The sorting should be done using bubble sort.

Input Format: The first line reads the number of elements in the array. The second line reads the array elements one by one.

Output Format: The output should be a sorted list.

For example:

Input	Result
6 3 4 8 7 1 2	1 2 3 4 7 8
5 4 5 2 3 1	1 2 3 4 5

Answer: (penalty regime: 0 %)

```
def bubble_sort(arr):
    n = len(arr)
    # Loop through all array elements
    for i in range(n):
        # Last i elements are already in place, no need to re-check them
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    # Read the number of elements (e.g., 6)
    n = int(input())

    # Read the array elements as a list of integers
    elements = list(map(int, input().split()))

    # Perform the bubble sort optimization
    sorted_elements = bubble_sort(elements)

    # Print the sorted list separated by spaces
    print(*(sorted_elements))


```

	Input	Expected	Got	
✓	6 3 4 8 7 1 2	1 2 3 4 7 8	1 2 3 4 7 8	✓
✓	6 9 18 1 3 4 6	1 3 4 6 9 18	1 3 4 6 9 18	✓
✓	5 4 5 2 3 1	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order*.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

For example:

Test	Result
`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`

Answer: (penalty regime: 0 %)

Reset answer

```
def sortedSquares(nums):
    # Square each element and sort the resulting array
    return sorted([x**2 for x in nums])

```

	Test	Expected	Got	
✓	`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`	`[0, 1, 9, 16, 100]`	✓
✓	`print(sortedSquares([-7,-3,2,3,11]))`	`[4, 9, 9, 49, 121]`	`[4, 9, 9, 49, 121]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program to find the k-th maximum value in a given list of integers. The program should handle various edge cases, including lists with duplicate values, empty lists, and invalid values of k. The k-th maximum value refers to the k-th largest distinct element in the list.

Background:

Finding the k-th maximum value in a list is a common problem in computer science, often encountered in fields like data analysis, competitive programming, and software development. This problem requires an understanding of sorting algorithms, data structures, and efficient problem-solving techniques. By solving this problem, one gains insights into how to handle large datasets and optimize performance in practical applications.

Problem Description:

Given a list of integers, the task is to determine the k-th maximum value in the list. The program should meet the following requirements:

1. **Input:**
    - A list of integers, which can contain both positive and negative values.
    - An integer k, representing the position of the maximum value to find.
2. **Output:**
    - The k-th maximum value in the list.
    - If k is greater than the number of distinct elements in the list or if the list is empty, the program should return an appropriate message indicating the error.

Constraints:

- The list may contain duplicate values.
- The value of k should be a positive integer.
- The list may contain up to 10^6 elements, and each element can be as large as 10^9 in magnitude.

Examples:

Consider the following examples for better understanding:

1. **Example 1:**
    - **Input:** list = [3, 1, 5, 4, 2], k = 2
    - **Output:** 4
    - **Explanation:** The distinct elements in the list are [1, 2, 3, 4, 5]. The 2nd maximum value is 4.
2. **Example 2:**
    - **Input:** list = [7, 7, 7, 7, 7], k = 1
    - **Output:** 7
    - **Explanation:** The distinct elements in the list are [7]. The 1st maximum value is 7.
3. **Example 3:**
    - **Input:** list = [2, 1, 2, 1, 2], k = 3
    - **Output:** -1
    - **Explanation:** The distinct elements in the list are [1, 2]. There is no 3rd maximum value.

For example:

Input	Result
5 3 1 5 4 2 2	4
6 7 7 7 7 7 7 1	7
10 2 1 2 1 2 1 2 1 2 1 3	-1

Answer: (penalty regime: 0 %)

```python
def find_kth_maximum(nums, k):
    # 1. Handle edge case: Empty list or invalid k
    if not nums or k <= 0:
        return -1

    # 2. Extract unique elements to handle duplicate values
    distinct_nums = list(set(nums))

    # 3. Handle edge case: k is greater than the number of unique elements
    if k > len(distinct_nums):
        return -1

    # 4. Sort the distinct elements in descending order
    distinct_nums.sort(reverse=True)

    # 5. Return the k-th maximum value (using 0-based indexing)
    return distinct_nums[k - 1]


# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    try:
        # Read the size of the array
        n = int(input())

        # Read the array elements as a list of integers
        nums = list(map(int, input().split()))

        # Read the value of k
        k = int(input())

        # Calculate and print the result
        print(find_kth_maximum(nums, k))
    except (IOError, ValueError):
        print(-1)
```

	Input	Expected	Got	
✓	5 3 1 5 4 2 2	4	4	✓
✓	6 7 7 7 7 7 7 1	7	7	✓
✓	10 2 1 2 1 2 1 2 1 2 1 3	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

To find the frequency of numbers in a list and display in sorted order.

Constraints:

1<=n, arr[i]<=100

Input:

1 68 79 4 90 68 1 4 5

output:

1 2

4 2

5 1

68 2

79 1

90 1

For example:

Input	Result
4 3 5 3 4 5	3 2 4 2 5 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

def display_frequency(arr):
    # 1. Count the frequency of each number
    counts = Counter(arr)

    # 2. Sort the unique keys in ascending order
    sorted_keys = sorted(counts.keys())

    # 3. Print each number and its frequency
    for key in sorted_keys:
        print(f"{key} {counts[key]}")

# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    try:
        # Read the entire line of elements as a list of integers
        arr = list(map(int, input().split()))

        # Display the sorted frequency mapping
        display_frequency(arr)
    except (IOError, ValueError):
        pass
```

	Input	Expected	Got	
✓	4 3 5 3 4 5	3 2 4 2 5 2	3 2 4 2 5 2	✓

	Input	Expected	Got	
✓	12 4 4 4 2 3 5	2 1 3 1 4 3 5 1 12 1	2 1 3 1 4 3 5 1 12 1	✓
✓	5 4 5 4 6 5 7 3	3 1 4 2 5 3 6 1 7 1	3 1 4 2 5 3 6 1 7 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an array of integers `arr`, replace each element with its rank.

The rank represents how large the element is. The rank has the following rules:

- Rank is an integer starting from 1.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Rank should be as small as possible.

Example 1:

```
Input: arr = [40,10,20,30]
Output: [4,1,2,3]
Explanation: 40 is the largest element. 10 is the smallest. 20 is the second smallest. 30 is the third smallest.
```

Example 2:

```
Input: arr = [100,100,100]
Output: [1,1,1]
Explanation: Same elements share the same rank.
```

Example 3:

```
Input: arr = [37,12,28,9,100,56,80,5,12]
Output: [5,3,4,2,8,6,7,1,3]
```

Constraints:

- $0 <= arr.length <= 10^5$
- $-10^9 <= arr[i] <= 10^9$

For example:

Test	Result
print(arrayRankTransform([40,10,20,30]))	[4, 1, 2, 3]

Answer: (penalty regime: 0 %)

Reset answer

```python
def arrayRankTransform(arr):
    # 1. Remove duplicates and sort the unique elements in ascending order
    sorted_unique = sorted(list(set(arr)))

    # 2. Create a dictionary mapping each number to its rank (1-indexed)
    #    Using enumerate starting at 1 automatically handles the "as small as possible" rule.
    rank_map = {num: rank for rank, num in enumerate(sorted_unique, 1)}

    # 3. Transform the original array using the dictionary lookup
    return [rank_map[x] for x in arr]

```

	Test	Expected	Got	
✓	`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`	`[4, 1, 2, 3]`	✓
✓	`print(arrayRankTransform([100,100,100]))`	`[1, 1, 1]`	`[1, 1, 1]`	✓
✓	`print(arrayRankTransform([37,12,28,9,100,56,80,5,12]))`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

The problem is that we want to reverse a array in O(N) linear time complexity and we want the algorithm to be in-place as well!
For example: input is [1,2,3,4,5] then the output is [5,4,3,2,1]
Input
5
1 2 3 4 5
Output
5 4 3 2 1

For example:

Input	Result
5 1 2 3 4 5	5 4 3 2 1

Answer: (penalty regime: 0 %)

```
def reverse_array_inplace(arr):
    left = 0
    right = len(arr) - 1

    # Swap elements from the outside moving inward
    while left < right:
        # Swap the elements in-place
        arr[left], arr[right] = arr[right], arr[left]
        # Move pointers closer together
        left += 1
        right -= 1

    return arr

# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    try:
        # Read the number of elements
        n = int(input())

        # Read the array elements as a list of integers
        arr = list(map(int, input().split()))

        # Reverse the array in-place
        reverse_array_inplace(arr)

        # Print the reversed array separated by spaces
        print(*(arr))
    except (IOError, ValueError):
        pass

```

	Input	Expected	Got	
✓	5 1 2 3 4 5	5 4 3 2 1	5 4 3 2 1	✓
✓	10 0 2 4 6 8 1 3 5 7 9	9 7 5 3 1 8 6 4 2 0	9 7 5 3 1 8 6 4 2 0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums`, return *an integer array* `counts` *where* `counts[i]` *is the number of smaller elements to the right of* `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

For example:

Test	Result
print(countSmaller([5,2,6,1]))	[2, 1, 1, 0]
print(countSmaller([-1]))	[0]

Answer: (penalty regime: 0 %)

Reset answer

```
def countSmaller(nums):
    n = len(nums)
    counts = [0] * n

    # Pair each number with its original index: (value, original_index)
    # This ensures we know which position in 'counts' to update.
    arr = [(val, idx) for idx, val in enumerate(nums)]

    def merge_sort(enum_arr):
        mid = len(enum_arr) // 2
        if mid:
            left = merge_sort(enum_arr[:mid])
            right = merge_sort(enum_arr[mid:])

            # Merge phase
            m, n_right = len(left), len(right)
            i = j = 0

            # Track how many elements from the right side are smaller
            # than the current element in the left side
            right_count = 0

            while i < m or j < n_right:
                # If right pointer has reached the end, or left element is smaller/equal
                if j == n_right or (i < m and left[i][0] <= right[j][0]):
                    enum_arr[i + j] = left[i]
                    # Update counts for the left item using its original index
                    counts[left[i][1]] += right_count
                    i += 1
                else:
                    enum_arr[i + j] = right[j]
                    # A smaller element from the right half is moving ahead
```

```
                    right_count += 1
                    j += 1

        return enum_arr

    merge_sort(arr)
    return counts

```

	Test	Expected	Got	
✓	print(countSmaller([5,2,6,1]))	[2, 1, 1, 0]	[2, 1, 1, 0]	✓
✓	print(countSmaller([50,20,60,10]))	[2, 1, 1, 0]	[2, 1, 1, 0]	✓
✓	print(countSmaller([-1]))	[0]	[0]	✓
✓	print(countSmaller([-1, -1]))	[0, 0]	[0, 0]	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an listof integers, sort the array in ascending order using the *Bubble Sort* algorithm above. Once sorted, print the following three lines:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

For example, given a worst-case but small array to sort: a=[6,4,1]. It took 3 swaps to sort the array. Output would be

```
Array is sorted in 3 swaps.
```

```
First Element: 1
```

```
Last Element: 6
```

Input Format

The first line contains an integer,n , the size of the list a .
The second line contains n, space-separated integers a[i].

Constraints

· 2<=n<=600

· 1<=a[i]<=$2x10^6$.

Output Format

You must print the following three lines of output:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

Sample Input 0

3

1 2 3

Sample Output 0

List is sorted in 0 swaps.

First Element: 1

Last Element: 3

For example:

Input	Result
3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3
5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9

Answer: (penalty regime: 0 %)

```
def bubble_sort_metrics(arr):
    n = len(arr)
    num_swaps = 0

    # Loop through all array elements
    for i in range(n):
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
                num_swaps += 1

    # 2. Print required metric reports
    print(f"List is sorted in {num_swaps} swaps.")
    print(f"First Element: {arr[0]}")
    print(f"Last Element: {arr[-1]}")

# --- Handling Input and Output to match your format ---
if __name__ == "__main__":
    try:
        # Read the number of elements
        n = int(input())

        # Read the array elements as a list of integers
        arr = list(map(int, input().split()))

        # Run bubble sort with metric counting
        bubble_sort_metrics(arr)
    except (IOError, ValueError):
        pass

```

	Input	Expected	Got	
✓	3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3	List is sorted in 3 swaps. First Element: 1 Last Element: 3	✓
✓	5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9	List is sorted in 4 swaps. First Element: 1 Last Element: 9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.